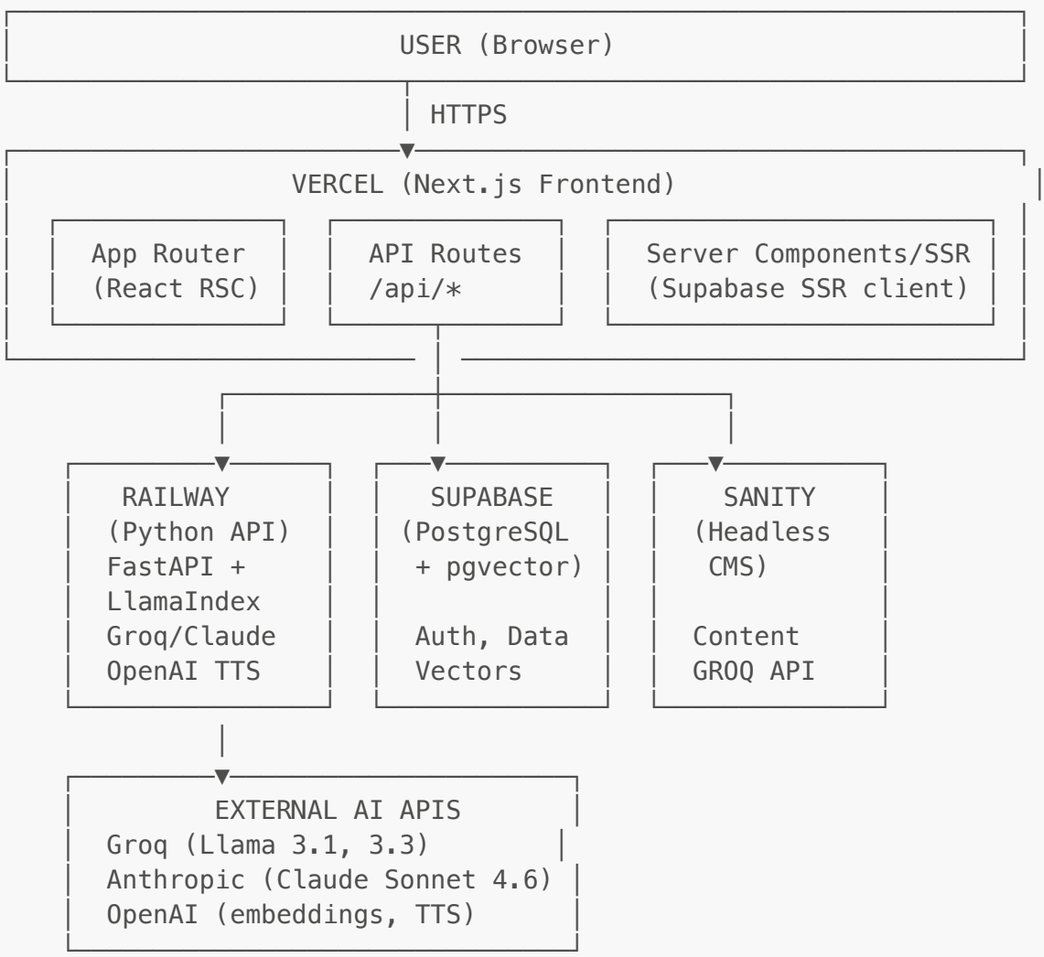
Technical Architecture — Vermont Health Platform (HTR)

Audience: Developers, architects, DevOps engineers. **Version**: 4.2.0

Table of Contents

1. System Overview



Request Flow — Chat

```
Browser → POST /api/chat (Next.js route)
  → Validates request schema (zod)
```

```
    → Forwards to Railway: POST /api/chat (FastAPI)
      → Verify JWT (Supabase JWT secret, HS256)
      → Determine user role → select LLM model
      → Embed query (OpenAI text-embedding-3-small)
      → Hybrid search (Supabase RPC: BM25 + pgvector)
      → Sentence window expansion (±3 sentences)
      → FlashRank re-ranking (ms-marco-MiniLM-L-12-v2)
      → Build context from top-5 nodes
      → Stream response via Groq/Anthropic
    ← Stream text/plain back to browser
  ← Rendered in right sidebar via streaming reader
```

---

2. Technology Stack

Frontend

Category	Technology	Version	Purpose
Framework	Next.js	16.1.6	App Router, SSR, API routes
Language	TypeScript	5.x	Type safety
UI	React	19.0.0	Component rendering
Styling	Tailwind CSS	4.0.0-beta.9	Utility-first CSS
Icons	@heroicons/react	2.2.0	SVG icon system
Markdown	react-markdown	10.1.0	CMS/AI content rendering
Maps	Leaflet, react-simple-maps	1.9.4, 1.0.0	Geographic visualizations
CMS Client	next-sanity	3.x	Sanity content fetching
DB Client	@supabase/ssr	0.9.0	Cookie-based Supabase sessions
Auth	@supabase/supabase-js	2.89.0	Client-side auth operations
Payments	@stripe/stripe-js, stripe	8.11.0, 20.4.1	Checkout and billing
Monitoring	@sentry/nextjs	9.47.1	Error tracking
Performance	web-vitals	5.1.0	Core Web Vitals reporting

Backend

Category	Technology	Version	Purpose
Framework	FastAPI	0.127.0	REST API + streaming
Server	Uvicorn	0.40.0	ASGI server
RAG Framework	LlamaIndex	0.14.10	Retrieval pipeline orchestration
LLM (free/sub)	Groq	—	Llama 3.1 8b / 3.3 70b
LLM (advisory)	Anthropic	—	Claude Sonnet 4.6
Embeddings	OpenAI	—	text-embedding-3-small
TTS	OpenAI	—	tts-1-hd
Re-ranking	FlashRank	0.2.9	ms-marco-MiniLM-L-12-v2 cross-encoder
DB Client	supabase-py	2.15.0	Database operations
Auth	PyJWT	2.10.0	JWT validation
Rate Limiting	slowapi	0.1.9	Per-endpoint rate limits

Category	Technology	Version	Purpose
HTTP	httpx	0.28.0	Async Sanity GROQ API calls
Monitoring	sentry-sdk	2.0	Error tracking

3. Repository Structure

```
Vermont-Health-Platform/
├── frontend/                    # Next.js application
│   ├── app/                     # App Router pages and API routes
│   │   ├── (auth)/              # Login, signup, password reset
│   │   ├── academy/             # Learning hub + all sub-tabs
│   │   ├── advisory/            # Advisory services
│   │   ├── advisory-hub/        # Alternative advisory entry point
│   │   ├── ahead-model/         # AHEAD CMS model
│   │   ├── api/                 # Next.js API route handlers (17 routes)
│   │   │   ├── chat/            # Proxy to Python backend
│   │   │   ├── personalized-learning/ # Curriculum generation proxy
│   │   │   ├── stripe/          # Checkout, portal, webhook
│   │   │   ├── ticker/          # Market/policy ticker data
│   │   │   └── ...
│   │   ├── california-calaim/   # CalAIM state initiative
│   │   ├── chat/                # Full-screen AI chat
│   │   ├── clinical/            # Clinical pillar + subcategories
│   │   ├── community/           # Community features
│   │   ├── connect-hub/         # Connect Hub
│   │   ├── dashboard/           # 50-state RHTP dashboard
│   │   │   └── [state]/         # Dynamic state profiles
│   │   ├── economics/           # Economics pillar
│   │   ├── equity/              # Equity pillar
│   │   ├── htr-simulator/       # 5-pillar scenario modeler
│   │   ├── hti-dashboard/       # HTI metrics dashboard
│   │   ├── multimedia/          # Video and media library
│   │   ├── policy/              # Policy pillar
│   │   ├── pricing/             # Subscription pricing
│   │   ├── research-lab/        # 19 analytical tools
│   │   ├── states/              # 50-state explorer
│   │   ├── technology/          # Technology pillar
│   │   ├── trending-topics/     # Real-time trending
│   │   ├── vermont-act-167/     # Vermont legislation
│   │   ├── layout.tsx           # Root layout + providers
│   │   ├── page.tsx             # Homepage
│   │   └── sitemap.ts           # Auto-generated sitemap
│   ├── components/              # Shared React components
│   │   ├── academy/             # Academy-specific components
│   │   │   ├── PersonalizedLearningHub.tsx
│   │   │   └── LearningTracksHub.tsx
│   │   ├── templates/           # Reusable page templates
│   │   │   └── HubPageTemplate.tsx
│   │   ├── AppShell.tsx         # Main layout container
│   │   ├── Breadcrumbs.tsx      # Dynamic breadcrumb trail
│   │   ├── CollapsibleSidebar.tsx # Sidebar animation wrapper
│   │   ├── CommandPalette.tsx   # ⌘K search overlay
│   │   ├── DarkModeToggle.tsx   # Theme switcher
│   │   ├── Footer.tsx           # Site footer
│   │   ├── Header.tsx           # Sticky header with mega-menus
│   │   ├── HeroCarousel.tsx     # Homepage hero carousel
│   │   ├── HomeContent.tsx      # Homepage feed and sections
│   │   ├── HomeSidebar.tsx      # Left navigation sidebar
│   │   ├── HubSubscribeCTA.tsx  # Per-pillar subscription CTA
│   │   ├── LatestHubReports.tsx # Per-pillar latest reports
│   │   ├── Logo.tsx             # SVG logo component
│   │   ├── NavDropdown.tsx      # Company nav dropdown (top bar)
```

```
│   │   ├── RightSidebar.tsx      # AI Analyst chat panel
│   │   ├── SidebarContext.tsx    # Left/right sidebar state
│   │   ├── TickerContext.tsx     # Ticker visibility state
│   │   ├── TickerStrip.tsx       # Scrolling headlines
│   │   └── ...
│   ├── lib/                      # Utilities, data, and DB clients
│   │   ├── auth.ts               # Server-side auth helpers
│   │   ├── sanity.ts             # Sanity client + urlFor
│   │   ├── advisory-data.ts      # Advisory services definitions
│   │   ├── db/
│   │   │   ├── client.ts         # Supabase anon + admin clients
│   │   │   └── rht-profiles.ts   # RHTP data fetchers
│   │   └── data/                 # Static data files
│   │       ├── hospital-data.ts
│   │       ├── learning-tracks-data.ts
│   │       ├── performance-index-data.ts
│   │       ├── rht-program.ts
│   │       └── state-initiatives-data.ts
│   ├── public/                   # Static assets
│   │   └── rhtp-icon.png
│   ├── next.config.js
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   └── package.json
│
├── backend/                      # FastAPI application
│   ├── routers/
│   │   ├── chat.py               # Chat + suggest endpoints
│   │   ├── ingest.py             # Index rebuild endpoints
│   │   ├── api_v1.py             # Developer API (key-authenticated)
│   │   └── personalized_learning.py # Curriculum generation + TTS
│   ├── services/
│   │   ├── auth.py               # JWT validation + role lookup
│   │   ├── db.py                 # Supabase singleton
│   │   ├── indexing.py           # Document ingestion pipeline
│   │   ├── llm.py                # LLM + FlashRank initialization
│   │   ├── retrieval.py          # HybridRetriever + re-ranking
│   │   └── tools.py              # Agentic tools (state metrics, lab finder)
│   ├── data/                     # PDF documents for indexing
│   ├── main.py                   # FastAPI app entry point
│   ├── config.py                 # Environment variable management
│   ├── requirements.txt
│   └── Procfile                  # Railway deployment command
│
└── docs/                         # Documentation (you are here)
    ├── README.md
    ├── user-guide.md
    ├── technical-architecture.md
    ├── developer-guide.md
    └── content-management.md
```

---

4. Frontend Architecture

App Router & Rendering Strategy

The frontend uses Next.js App Router with a mix of Server Components (RSC) and Client Components:

Page Type	Rendering	Reason
Pillar hub pages (`/policy`, etc.)	Server Component	Static content, SEO
Homepage (`/`)	Server Component	Sanity data fetching at request time

Page Type	Rendering	Reason
Dashboard (`/dashboard`)	Server Component + Client	Initial auth check server-side, chart interactions client-side
AI Analyst chat	Client Component	Streaming, real-time state
Academy learning	Client Component	localStorage, quiz state, audio
HTR Simulator	Client Component	Interactive form state
Research Lab	Client Component (gated)	Role check server, tool UIs client

Layout System

The root layout (`app/layout.tsx`) wraps everything in a provider chain:

```
ThemeProvider                  ← dark/light mode
└── WebVitalsReporter          ← performance monitoring
└── CommandPalette             ← global ⌘K search
└── TickerProvider             ← ticker strip visibility state
    └── SidebarProvider        ← left/right sidebar open state
        └── Header             ← sticky header (reads sidebar context)
        └── AppShell           ← content + sidebars layout
            └── CollapsibleSidebar (left) ← HomeSidebar
            └── main           ← page content
            └── CollapsibleSidebar (right) ← RightSidebar (AI Analyst)
            └── Footer
```

AppShell Sidebar Behavior

`AppShell.tsx` manages when sidebars show/hide:

- **Always hidden**: `/studio` (Sanity CMS admin), `/chat` (full-width chat)
- **Persists user preference**: All other pages — if you open the sidebar on the policy page, it stays open when you navigate to economics
- **Auto-collapse on mobile**: `< 1024px` for left sidebar, `< 1280px` for right sidebar
- **Floating Ask AI button**: Appears on desktop when right sidebar is closed

Header Mega-Menu System

The header (`Header.tsx`) implements a hover-activated mega-menu pattern:

- `activeMegaMenu` state tracks which panel is open (`null | "intelligence" | "learn" | "analyze" | "advise"`)
- `openMegaMenu()` / `closeMegaMenu()` use a 150ms timeout to prevent accidental closure
- `cancelClose()` resets the timeout when the cursor moves back onto the panel
- Each panel renders as a full-width absolute-positioned `div` below the nav bar
- Closes automatically on route change via `usePathname()` effect

Context Providers

Context	File	State
`SidebarContext`	`components/SidebarContext.tsx`	`isLeftOpen`, `isRightOpen`, toggle/set functions
`TickerContext`	`components/TickerContext.tsx`	`isHeaderVisible`, `isStripVisible` (two separate ticker zones)

Both are consumed by `Header.tsx` and `AppShell.tsx` to coordinate sidebar state across the header buttons and the sidebar wrappers without prop drilling.

Supabase SSR Pattern

The frontend uses `@supabase/ssr` for cookie-based session management (required by Next.js App Router):

```
// Server Component usage (lib/auth.ts)
const supabase = createSupabaseServerClient(); // reads cookies
const { data: { user } } = await supabase.auth.getUser();

// Client Component usage (lib/db/client.ts)
export const db = createBrowserClient(url, anonKey);
```

Server components use `createServerClient` (reads from Next.js `cookies()`); client components use `createBrowserClient`. Never use the server client in client components.

---

5. Backend Architecture

FastAPI Application (`backend/main.py`)

Version: 4.2.0

Startup sequence:

1. Initialize Sentry (if `SENTRY_DSN` set)
2. Configure CORS (allows `FRONTEND_URL` + localhost:3000)
3. Initialize global LlamaIndex settings (LLM + embedding models)
4. Warm up FlashRank re-ranker (downloads model on first run)
5. Load or build vector index:
    - If Supabase available: load from `rag_documents` pgvector table
    - If unavailable: load from local `storage/` JSON fallback
    - If no index exists: build fresh from Sanity + PDFs

Registered routers:

- `chat.router` — `/api/chat`, `/api/suggest`
- `ingest.router` — `/api/ingest`, `/api/ingest/status`
- `api_v1.router` — `/api/v1/states`, `/api/v1/survey/results`
- `personalized_learning.router` — `/api/personalized-learning/generate`, `/api/personalized-learning/tts`

Health endpoint: `GET /health` → `{ status, version, capabilities, environment }`

Router Details

`routers/chat.py`

```
POST /api/chat
  Auth: Bearer JWT (require_subscriber)
  Rate: 30/min
  Body: { message, history[], temperature?, systemPrompt? }
  Response: text/plain streaming

POST /api/suggest
  Auth: optional
  Rate: 60/min
  Body: { topic }
  Response: JSON array of 3 suggested questions
```

Chat pipeline decision tree:

```
Is user advisory/admin?
├── YES → ReActAgent (LlamaIndex)
│         Tools: query_state_metrics, list_research_lab_tools
│         Max iterations: 5
│         Model: Claude Sonnet 4.6
└── NO  → ContextChatEngine (RAG-only)
          Model: Groq (8b or 70b by role)
```

Prompt injection detection: Scans incoming messages for 13 known leaked-prompt phrases. Rejects with 400 if detected.

Conversation persistence: All messages stored to `conversations` and `conversation_messages` tables (skipped in dev mode).

routers/ingest.py

```
POST /api/ingest
  Auth: Bearer INGEST_SECRET
  Rate: 5/hour
  Response: 202 Accepted + job_id

GET /api/ingest/status
  Auth: Bearer INGEST_SECRET
  Response: { status, started_at, completed_at, error_message }
```

Single-job queue: If an ingest is already running, new requests return 409 Conflict.

routers/api_v1.py

```
GET /api/v1/states
  Auth: X-HTR-API-Key header
  Response: { data: RhtStateProfile[], count: N }

GET /api/v1/states/{state_id}
  Auth: X-HTR-API-Key header
  Response: { data: RhtStateProfile }

GET /api/v1/survey/results
  Auth: X-HTR-API-Key header
  Response: { data: SurveyAggregate }
```

API key validation: SHA256(incoming_key) matched against `key_hash` in `api_keys` table.

routers/personalized_learning.py

```
POST /api/personalized-learning/generate
  Auth: Bearer JWT
  Body: { role, topics[], difficulty, weekly_hours, goals, format_preferences[]
}
  Response: JSON curriculum object

POST /api/personalized-learning/tts
  Auth: Bearer JWT
  Body: { text, voice }
  Response: audio/mpeg stream (MP3)
```

Curriculum generation pipeline:

1. Fetch live Sanity catalog (academy modules, case studies by pillar)
2. Build topic-to-pillar and topic-to-track mappings
3. Embed catalog in prompt to prevent hallucinated links
4. Call Groq Llama 3.3 70b (temp=0.65, max_tokens=8000)
5. Validate all `platform_link` values against actual Sanity slugs
6. Generate relevance bridges (Groq, temp=0.25)
7. Return structured JSON

Service Layer Details

`services/retrieval.py` — HybridRetriever

```
class HybridRetriever:
    def retrieve(query: str, top_k: int = 5) -> List[NodeWithScore]:
        1. embed(query) → vector via OpenAI
        2. supabase.rpc("hybrid_search_rag", {
               query_embedding: vector,
               query_text: query,
               match_count: top_k * 4  # over-fetch for re-ranking
           })
           # RPC performs: BM25 + cosine ANN → RRF merge
        3. expand sentence windows (±3 sentences)
        4. flashrank.rerank(query, nodes) → top_k final nodes
        return nodes
```

The Supabase RPC function `hybrid_search_rag` must exist in the database. It combines:

- **Dense search**: `embedding <=> query_embedding` (cosine distance via pgvector)
- **Sparse search**: `to_tsvector(content) @@ plainto_tsquery(query_text)` (BM25 via PostgreSQL FTS)
- **Fusion**: Reciprocal Rank Fusion (RRF) merges the two ranked lists

`services/llm.py` — Model Router

```
def get_llm_for_role(role: str) -> LLM:
    if role in ("free", "student"):
        return Groq(model="llama-3.1-8b-instant", ...)
    elif role in ("subscriber", "professional"):
        return Groq(model="llama-3.3-70b-versatile", ...)
    else:  # advisory, admin
        return Anthropic(model="claude-sonnet-4-6", ...)
```

`services/indexing.py` — Document Pipeline

```
async def build_index():
    # 1. Load PDFs from backend/data/
    pdf_docs = SimpleDirectoryReader("data/").load_data()

    # 2. Fetch Sanity content via GROQ API
    sanity_docs = await fetch_sanity_content()  # 8 content types

    # 3. Parse into nodes with sentence windows
    nodes = SentenceWindowNodeParser(window_size=3).get_nodes_from_documents(
        pdf_docs + sanity_docs
    )
```

```
    # 4. Embed + store
    if supabase_available:
        index = VectorStoreIndex(nodes, vector_store=PGVectorStore(...))
    else:
        index = VectorStoreIndex(nodes)  # local JSON storage
```

---

6. AI Pipeline

RAG Pipeline (Detailed)

```
User Query: "What are the financial implications of global budgets for CAHs?"

Step 1 — Embedding
  query → OpenAI text-embedding-3-small → 1536-dim vector

Step 2 — Hybrid Retrieval (Supabase RPC)
  Dense:  cosine ANN on rag_documents.embedding
  Sparse: tsvector BM25 on rag_documents.content
  Merge:  Reciprocal Rank Fusion → top 20 candidates

Step 3 — Sentence Window Expansion
  For each candidate node:
    original chunk + 3 sentences before + 3 sentences after
  (increases context, reduces truncation artifacts)

Step 4 — FlashRank Re-ranking
  Model: ms-marco-MiniLM-L-12-v2 (cross-encoder)
  Input: [(query, expanded_node_text)] × 20
  Output: relevance scores → sort → take top 5

Step 5 — Context Assembly
  Format top-5 nodes as context block
  Append to system prompt with source metadata

Step 6 — LLM Generation
  Subscriber: Groq llama-3.3-70b-versatile
  System prompt: HTR analyst persona + context block
  Stream: yes (text/plain, chunked transfer encoding)

Step 7 — Logging
  Log to rag_query_log: query, doc_ids, scores, model, latency_ms
```

Agentic Pipeline (Advisory/Admin)

Advisory and Admin users get a ReActAgent instead of ContextChatEngine:

```
User: "Compare Vermont and New Hampshire's RHTP performance"

→ ReActAgent (max 5 iterations)

Iteration 1:
  Think: I need Vermont metrics
  Act: query_state_metrics("vermont")
  Observe: { overall: 42, quality: 38, equity: 44, ... }

Iteration 2:
  Think: I need New Hampshire metrics
  Act: query_state_metrics("new-hampshire")
  Observe: { overall: 61, quality: 67, equity: 55, ... }

Iteration 3:
```

```
  Think: I have enough data to answer
  Act: [final answer with comparison]
```

Available tools:

Tool	Function	Input	Output
`query_state_metrics`	`services/tools.py`	`state: str`	Performance index dict
`list_research_lab_tools`	`services/tools.py`	`topic: str`	Markdown list with `/research-lab/*` links

Personalized Learning Generation

```
Input: { role: "Hospital Administrator", topics: ["Value-Based Care", "SDOH"],
         difficulty: "intermediate", weekly_hours: "3-5", goals: "..." }

Step 1 — Catalog Fetch
  GET Sanity: academyModule[], caseStudy[] (by pillar)

Step 2 — Topic Mapping
  "Value-Based Care" → pillar: Economics
  "SDOH" → pillar: Equity

Step 3 — Prompt Construction
  System: "You are an expert curriculum designer..."
  Context: Live Sanity catalog with real slugs embedded
  Instructions: Generate N-week curriculum, validate all platform_links

Step 4 — Generation
  Model: Groq llama-3.3-70b-versatile
  Temperature: 0.65, max_tokens: 8000
  Output: Structured JSON (weeks → themes → items)

Step 5 — Validation
  For each item.platform_link:
    Assert slug exists in Sanity catalog
    Replace hallucinated links with closest real match

Step 6 — Relevance Bridges
  For each case_study item:
    Generate 2-3 sentence personalized explanation
    Model: Groq llama-3.3-70b-versatile, temperature: 0.25
```

---

7. Authentication & Authorization

JWT Flow

```
1. User logs in via Supabase Auth (email/password or OAuth)
2. Supabase issues JWT signed with SUPABASE_JWT_SECRET (HS256)
3. JWT stored in httpOnly cookie by @supabase/ssr
4. Every SSR page: createServerClient reads cookie → getUser()
5. Every API call to Python backend:
   Next.js API route forwards Authorization: Bearer <jwt>
6. Python backend: PyJWT.decode(token, SUPABASE_JWT_SECRET, algorithms=
["HS256"])
```

Role Hierarchy

```
admin
  └── advisory
        └── professional
              └── subscriber
                    └── student
                          └── free
```

Roles are stored in `user_roles` table (one row per role assignment). The auth service fetches all rows for the user and returns the highest role in the hierarchy.

Frontend Auth Helpers (`lib/auth.ts`)

```
// Use in Server Components and API Routes only
await getUser()             // → AuthUser | null
await requireAuth()         // → AuthUser (redirects to /login if not
authenticated)
await requireRole("subscriber")  // → AuthUser (redirects to /upgrade if
insufficient role)
await canAccess("advisory") // → boolean (no redirect)
roleAtLeast("subscriber", userRole) // → boolean comparison
```

Backend Auth Dependencies (`services/auth.py`)

```
# FastAPI dependency injection
async def get_auth_user(request: Request) -> AuthUser
async def require_subscriber(user = Depends(get_auth_user)) -> AuthUser

# Dev mode (SUPABASE_JWT_SECRET not set)
# All requests → user_id="dev", role="subscriber"
```

Developer API Auth

```
Client → GET /api/v1/states
         Header: X-HTR-API-Key: htr_<32-hex-chars>

Backend:
  key_hash = SHA256(api_key)
  SELECT * FROM api_keys WHERE key_hash = ? AND revoked_at IS NULL
  IF daily_limit exceeded → 429 Too Many Requests
  INCREMENT requests_today
```

Ingest Endpoint Auth

```
POST /api/ingest
Header: Authorization: Bearer <INGEST_SECRET>

INGEST_SECRET is a shared secret stored in:
  - Backend .env as INGEST_SECRET
  - Frontend .env as INGEST_SECRET (for webhook forwarding)
```

---

8. Database Schema

All tables are in Supabase PostgreSQL. Row-Level Security (RLS) is enabled on user-facing tables.

auth.users (Supabase managed)

```
id          UUID PRIMARY KEY
email       TEXT
created_at  TIMESTAMPTZ
```

profiles

```
id          UUID PRIMARY KEY REFERENCES auth.users(id)
full_name   TEXT
avatar_url  TEXT
created_at  TIMESTAMPTZ
```

RLS: Users can only read/write their own row.

user_roles

```
id         UUID PRIMARY KEY
user_id    UUID REFERENCES auth.users(id)
role       TEXT CHECK (role IN
('free','subscriber','student','professional','advisory','admin'))
granted_at TIMESTAMPTZ
granted_by UUID
```

RLS: Users can read their own roles. Admins can write.

subscriptions

```
id                 UUID PRIMARY KEY
user_id            UUID REFERENCES auth.users(id)
stripe_customer_id TEXT
stripe_sub_id      TEXT
plan               TEXT
status             TEXT
current_period_end TIMESTAMPTZ
created_at         TIMESTAMPTZ
```

RLS: Users can read their own subscription.

conversations

```
id         UUID PRIMARY KEY
user_id    UUID REFERENCES auth.users(id)
title      TEXT
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

conversation_messages

```
id              UUID PRIMARY KEY
conversation_id UUID REFERENCES conversations(id)
role            TEXT CHECK (role IN ('user','assistant'))
```

```
content         TEXT
created_at      TIMESTAMPTZ
```

rag_documents

```
id          BIGSERIAL PRIMARY KEY
content     TEXT
metadata    JSONB        -- title, source_type, pillar, url, chunk_index
embedding   VECTOR(1536) -- OpenAI text-embedding-3-small
created_at  TIMESTAMPTZ
```

Index: `embedding vector_cosine_ops` (pgvector IVFFlat or HNSW) Full-text: `to_tsvector(content)` GIN index for BM25

rag_query_log

```
id                UUID PRIMARY KEY
user_id           TEXT
query             TEXT
role              TEXT
model_used        TEXT
retrieved_doc_ids TEXT[]
retrieved_scores  FLOAT[]
response_preview  TEXT
latency_ms        INTEGER
was_zero_result   BOOLEAN
created_at        TIMESTAMPTZ
```

api_keys

```
id              UUID PRIMARY KEY
user_id         UUID REFERENCES auth.users(id)
key_hash        TEXT UNIQUE  -- SHA256(raw_key)
tier            TEXT         -- researcher, enterprise
requests_today  INTEGER DEFAULT 0
last_used_at    TIMESTAMPTZ
revoked_at      TIMESTAMPTZ
created_at      TIMESTAMPTZ
```

ingest_jobs

```
id            UUID PRIMARY KEY
status        TEXT CHECK (status IN ('queued','running','completed','failed'))
started_at    TIMESTAMPTZ
completed_at  TIMESTAMPTZ
error_message TEXT
```

state_performance_index

```
id                  UUID PRIMARY KEY
state_name          TEXT
state_id            TEXT UNIQUE
performance_score   FLOAT
cost_index          FLOAT
```

```
  quality_score       FLOAT
  access_score        FLOAT
  equity_score        FLOAT
  innovation_score    FLOAT
  preventive_care_rate FLOAT
  uninsured_rate      FLOAT
  data_year           INTEGER
  updated_at          TIMESTAMPTZ
```

rht_state_profiles

```
  id               UUID PRIMARY KEY
  state_id         TEXT UNIQUE
  state_name       TEXT
  award_amount     TEXT
  status           TEXT
  strategic_focus  TEXT
  description      TEXT
  initiatives      JSONB  -- Array of {title, description, status}
  metrics          JSONB  -- Array of {label, value, trend, status, target}
  simulation       JSONB  -- Optional simulation data
```

survey_aggregate

```
  id         UUID PRIMARY KEY
  edition    TEXT
  results    JSONB
  created_at TIMESTAMPTZ
```

Required PostgreSQL Function

```
  -- Hybrid search combining BM25 + vector ANN
  CREATE OR REPLACE FUNCTION hybrid_search_rag(
    query_embedding VECTOR(1536),
    query_text      TEXT,
    match_count     INT DEFAULT 20
  ) RETURNS TABLE (
    id       BIGINT,
    content  TEXT,
    metadata JSONB,
    score    FLOAT
  ) LANGUAGE plpgsql AS $$
    -- Implementation: RRF merge of dense + sparse results
  $$;
```

---

9. Content Management System

See Content Management Guide for the full Sanity CMS reference.

Content Types Indexed by the RAG System

Sanity Type	GROQ Field	Indexed Fields
policyAnalysis	title, body, pillar, publishedAt	Full body text
post	title, body, categories	Full body text

Sanity Type	GROQ Field	Indexed Fields
`academyModule`	title, content, track, difficulty	Content text
`caseStudy`	title, summary, body, pillar	Full body text
`definition`	term, definition	Definition text
`analystNote`	headline, content, author	Content text
`webinar`	title, description, transcript	Description + transcript
`report`	title, summary, body, pillar	Full body text

Content Sync Webhook

```
Sanity publishes content
  → Webhook POST /api/webhooks/sanity (Next.js)
    → Validates SANITY_WEBHOOK_SECRET
    → POST /api/ingest (FastAPI)
      → Queues index rebuild job
      → Rebuilds rag_documents table
      → New content searchable within ~2–5 minutes
```

---

10. Payment System

Stripe Integration

Frontend routes:

- `POST /api/stripe/checkout` — creates Stripe checkout session, redirects to Stripe
- `POST /api/stripe/team-checkout` — creates team/org checkout session
- `POST /api/stripe/portal` — creates Stripe billing portal session URL
- `POST /api/stripe/webhook` — receives Stripe events, updates `subscriptions` table

Webhook events handled:

- `checkout.session.completed` → create subscription record, grant role
- `customer.subscription.updated` → update plan/status
- `customer.subscription.deleted` → downgrade to free role
- `invoice.payment_failed` → flag subscription as past_due

Stripe products (configured in Stripe dashboard):

- Subscriber monthly
- Subscriber annual
- Professional monthly
- Professional annual

Price IDs stored as env vars (`STRIPE_PRICE_SUBSCRIBER_MONTHLY`, etc.).

---

11. Observability & Monitoring

Sentry

Both frontend and backend integrate Sentry:

```
Frontend: @sentry/nextjs
  - Automatic error capture for API routes and React components
  - Performance tracing for page loads and API calls
  - Source map upload via SENTRY_AUTH_TOKEN
```

```
Backend: sentry-sdk
  - FastAPI middleware integration
  - Exception capture with user context (user_id, role)
  - Environment tagging (development/production)
```

Configure via: `SENTRY_DSN` (both), `SENTRY_AUTH_TOKEN` (frontend build only).

Web Vitals

`WebVitalsReporter` component in root layout captures Core Web Vitals (LCP, FID, CLS, TTFB, FCP) and can report to Sentry or a custom analytics endpoint.

RAG Query Log

Every chat query is logged to `rag_query_log`:

- Enables offline evaluation of retrieval quality
- Track `was_zero_result` to find gaps in knowledge base
- Compare `retrieved_scores` across model changes
- Monitor `latency_ms` for performance regressions

---

12. Security Architecture

Transport Security

- All traffic over HTTPS (Vercel + Railway enforce TLS)
- CORS: Backend only accepts requests from `FRONTEND_URL` and `localhost:3000`

Authentication Security

- JWTs signed HS256 with Supabase JWT secret (never exposed to browser)
- httpOnly cookies (XSS-resistant session storage)
- Short JWT expiry with Supabase refresh token rotation

Input Validation

- All API route inputs validated with Zod schemas (frontend) and Pydantic (backend)
- Max message length: 2000 chars (user input), 4000 chars (history items)
- System prompt max: 800 chars
- Temperature clamped: 0.0–1.0

Prompt Injection Protection

- Backend scans incoming messages for 13 known prompt injection phrases
- Returns 400 Bad Request if detected
- System prompt never exposed to user-controlled input

Rate Limiting

- `/api/chat`: 30/min (slowapi, per IP)
- `/api/suggest`: 60/min
- `/api/ingest`: 5/hour
- Developer API: 1000/day (researcher), unlimited (enterprise)

Database Security

- Row-Level Security (RLS) on all user-facing tables
- Service role key only used server-side, never in browser
- Anon key restricted by RLS policies
- pgvector table not directly accessible from browser

13. Performance Considerations

Frontend

- Next.js App Router enables selective hydration (only interactive components ship JS)
- Server Components avoid unnecessary client-side JS for static content (pillar pages, about pages)
- Sanity CDN disabled (`useCdn: false`) for real-time content; consider enabling for heavily trafficked static pages
- Images use Next.js `<Image>` for automatic optimization and WebP conversion
- Tailwind CSS purges unused styles at build time

Backend

- FlashRank re-ranker loaded once at startup (lazy init on first request), then cached in memory
- Supabase connection pooling recommended for production (PgBouncer)
- Index built once and reused across requests; only rebuilt via `/api/ingest`
- Streaming responses (no blocking wait for full LLM output)
- Groq API offers very low latency for Llama models (~200ms TTFT)

Caching Opportunities

- Pillar hub pages: could be cached at Vercel edge (currently ISR-eligible)
- Sanity content: currently `revalidate: 60` on homepage query
- State performance data: infrequently changing, safe to cache aggressively

---

14. Component Map

Key Page Components

Page	File	Rendering	Notes
Homepage	`app/page.tsx` + `HomeContent.tsx`	Server + Client	Sanity data server-side, filters client-side
Policy Hub	`app/policy/page.tsx`	Server	Static + LatestHubReports
Academy	`app/academy/page.tsx`	Server	Tabs rendered via HubPageTemplate
Personalized Learning	`components/academy/PersonalizedLearningHub.tsx`	Client	localStorage, API calls, audio
Research Lab	`app/research-lab/page.tsx`	Server (gated)	Role check → ResearchLabHub client component
HTR Simulator	`app/htr-simulator/page.tsx`	Client	5-pillar form state
Dashboard	`app/dashboard/page.tsx`	Server + Client	Map visualization
State Profile	`app/dashboard/[state]/page.tsx`	Server	Dynamic segment, Supabase fetch
Chat	`app/chat/page.tsx`	Client	Full-screen, no sidebars

Shared Layout Components

Component	Responsibility	Key Props/State
`Header.tsx`	Navigation, search, mega-menus, ticker	`activeMegaMenu`, `searchQuery`, sidebar context
`AppShell.tsx`	Content area layout, sidebar orchestration	`isLeftOpen`, `isRightOpen`, floating Ask AI
`HomeSidebar.tsx`	Left nav (persistent, context-aware)	`expandedPillars` accordion state
`CollapsibleSidebar.tsx`	Animated sidebar wrapper	`isOpen`, `side`, `stickyTop`
`RightSidebar.tsx`	AI Analyst chat panel	`messages[]`, `isLoading`, streaming reader
`Breadcrumbs.tsx`	Current location trail	`pathname`, `searchParams` (tab labels)
`HeroCarousel.tsx`	Homepage hero with auto-advance	`currentIndex`, `isPaused`, RHTP modal
`HomeContent.tsx`	Homepage sections (Wire, Capabilities, Trending)	`pillarFilter`, `typeFilter`
`CommandPalette.tsx`	⌘K global search overlay	Global keyboard listener
`TickerStrip.tsx`	Scrolling headlines	`tickerData`, `isVisible`, `theme`